Technical Notes: Verilog RTL and RISC-V Practice Activities

Day 1: RTL building blocks and self-checking verification

1.1 Parameterized ALU

- **Approach:** One module with parameter WIDTH = 32. ADD, SUB and signed SLT share a single adder: the second operand is inverted and the carry-in set for subtraction. Shifts use only the low $clog2(WIDTH) bits of b. Flags: zero, negative, carry (carry-out of a + ~b + 1, i.e. not-borrow, for SUB) and signed overflow. Every output has a default value so no latches are inferred.
- **Verification:** Self-checking testbench with a reference model inside it. About 25 directed corner cases (zero, maximum positive, minimum negative, carry, overflow, shift by 0 and 31) plus 500 random vectors. Each vector compares the result and all four flags with !==.
- **Result:** 525 vectors, 0 FAIL, final line ALL TESTS PASSED.
- **Issues:** vsim first failed with "design unit alu not found" because alu.v had not compiled (unfinished expressions). Fixed by completing the file and compiling both files with vlog before vsim. Signed overflow and signed SLT needed sign-aware logic, so the directed tests include boundary values.

1.2 Up/down counter

- **Approach:** 8-bit counter with synchronous reset, enable, up/down control and parallel load. Priority order: reset, then load, then count. Non-blocking assignments in a clocked block.
- **Verification:** Directed checks of reset, enable, count-up, count-down and parallel load.
- **Result:** All checks passed.
- **Issues:** No significant issues recorded.

1.3 Barrel shifter

- **Approach:** 32-bit combinational shifter with logical left, logical right and arithmetic right modes, shift amount 0 to 31 in one cycle. Arithmetic right uses $signed.
- **Verification:** Loop over all 32 shift amounts in all three modes against a reference model in the testbench.
- **Result:** 96 checks passed, 0 FAIL.
- **Issues:** No significant issues recorded. Design note: arithmetic right shift needs a signed operand, otherwise it behaves as a logical shift.

1.4 Register file

- **Approach:** 32 x 32 array with two asynchronous read ports and one synchronous write port. Reads of x0 return zero and writes to x0 are ignored. A bypass returns the write data when a read and a write hit the same register in the same cycle.
- **Verification:** Directed tests (x0 write, bypass, back-to-back writes) and a random test against a software model: 200 random operations plus 4 directed operations, 204 in total.
- **Result:** All 204 operations passed.
- **Issues:** No significant issues recorded.

1.5 Ripple-carry adder/subtractor

- **Approach:** 8-bit adder built from full_adder instances in a generate loop. A mode input XORs b and feeds the first carry-in, giving two's-complement subtraction. Signed overflow is carry-in XOR carry-out of the top bit.
- **Verification:** 4,000 random vectors compared with the behavioural + and - operators, including carry and overflow.
- **Result:** 4,000 vectors passed.
- **Issues:** No significant issues recorded.

1.6 Priority encoder and decoder

- **Approach:** 8-to-3 priority encoder with a valid output, written once with casez and once with an if-else chain. 3-to-8 decoder with an enable input.

- **Verification:** All 256 encoder inputs, and all decoder codes with enable on and off. Both encoder versions were compared with each other.
- **Result:** Both encoders matched on every input.
- **Issues:** No significant issues recorded.

1.7 Shift register and LFSR

- **Approach:** 8-bit universal shift register (hold, shift left, shift right, parallel load). 8-bit LFSR with a seed input and feedback taps chosen for a maximal-length sequence.
- **Verification:** Shift register modes checked individually. The LFSR testbench counts states and checks that it visits 255 nonzero states, never hits all-zero, and then repeats.
- **Result:** 255-state period confirmed.
- **Issues:** The LFSR must start from a nonzero seed because all-zero is a lock-up state for this feedback.

1.8 Edge detector and debouncer

- **Approach:** Two-flop synchronizer on the input, a counter-based debounce filter (output changes only after the input is stable for a threshold count), and a rising-edge detector producing a one-cycle pulse.
- **Verification:** A noisy input that bounces several times before settling high; the testbench checks that exactly one pulse appears after the input is stable.
- **Result:** One clean pulse, no spurious pulses.
- **Issues:** Debounce timing is set by the counter threshold, so the testbench uses a small threshold to keep simulation short.